

Shah & Anchor Kutchhi Engineering College (SAKEC)
Department of Computer Engineering
M.Tech (Computer Engineering)

Q&A Based Interactive Storytelling Using Generative AI

Anjali Madan Jha

Roll No: 124MTCM1008
M.Tech Computer Engineering, SAKEC

Guide: Dr. Vidyullata Devmane

Professor, Department of Computer Engineering, SAKEC

April 2026

ABSTRACT

This paper presents the design, implementation, and evaluation of a Q&A Based Interactive Storytelling System powered by Generative AI. The system employs a structured Question-Answer dialogue framework consisting of 17 questions across four narrative dimensions (setting, characters, themes, plot) to elicit comprehensive user preferences. These preferences are processed through a seven-module pipeline comprising context extraction, knowledge memory, prompt engineering, and story flow management to generate personalized, coherent narratives using Large Language Models (LLMs).

The system implements a five-phase story progression model based on Freytag's pyramid (Introduction, Rising Action, Climax, Falling Action, Resolution) with word-count-driven phase transitions. A provider-agnostic LLM integration layer supports four providers: Ollama (local), Groq (free cloud), Hugging Face (free), and OpenAI (paid), enabling flexible deployment. Active consistency checking monitors character continuity, plot thread resolution, and narrative pacing.

Evaluation across 50 generated stories and 15 user participants demonstrates: average narrative quality scores of 4.1/5.0 (coherence), 4.1/5.0 (creativity), 4.4/5.0 (engagement), and 3.8/5.0 (consistency) on Likert scales; Groq cloud inference achieving 1.8s average initial generation time compared to 12.4s for local Ollama; and an overall user satisfaction score of 4.3/5.0. The consistency checking module achieves 87% detection rate for character contradictions with an 8% false positive rate.

Keywords: *Interactive Storytelling, Generative AI, Large Language Models, Natural Language Processing, Prompt Engineering, Narrative Generation, Human-AI Co-creation, FastAPI, LLaMA, Groq*

I. INTRODUCTION

A. Background and Motivation

Interactive storytelling represents a compelling intersection of artificial intelligence, natural language processing, and creative computing. Traditional story generation systems relied on rule-based approaches, template filling, or planning-based methods that produced rigid, formulaic narratives [1][2][3]. The advent of Large Language Models (LLMs) such as GPT-4 [13], LLaMA [11], and Mistral [12] has dramatically expanded possibilities for automated narrative generation, enabling systems to produce fluent, contextually rich text approaching human creative writing quality.

However, a fundamental challenge persists: generating stories that align with individual user preferences while maintaining narrative coherence across extended passages. Most existing systems either provide limited user control through single-prompt generation or overwhelm users with complex interfaces requiring creative writing expertise. This research addresses these gaps by proposing a structured Q&A-driven approach that makes personalized storytelling accessible to all users.

B. Problem Statement

Current AI story generation tools face several critical limitations: (1) limited user personalization beyond a single text prompt, (2) lack of narrative structure management leading to incoherent long-form content, (3) inability to maintain character and plot consistency across story segments, (4) rigid deployment requirements tied to specific cloud APIs, and (5) high operational costs requiring paid subscriptions. This research addresses these gaps comprehensively.

C. Research Objectives

The primary objectives of this research are:

- Design a structured Q&A framework capturing comprehensive narrative preferences across four dimensions without requiring creative writing expertise.
- Develop a seven-module pipeline architecture transforming user responses into contextually rich, personalized stories using LLMs.
- Implement a five-phase story flow management system ensuring narrative coherence through word-count-driven phase transitions.
- Support multiple LLM providers (local and cloud-based) for flexible, cost-effective deployment.

- Enable interactive story continuation through AI-generated meaningful choice options.
- Evaluate the system's narrative quality, consistency checking accuracy, and user satisfaction through structured experiments.

D. Scope of Work

This work encompasses the complete design, implementation, and evaluation of a web-based interactive storytelling system. The technology stack includes FastAPI (backend), HTML5/CSS3/JavaScript (frontend), SQLite3 (persistence), and httpx (async LLM communication). The system handles 17 structured questions across 4 narrative dimensions and supports story generation, continuation, export, and session management.

II. LITERATURE REVIEW

A. Traditional Approaches to Story Generation

Early computational storytelling systems employed rule-based and planning approaches. TALE-SPIN (Meehan, 1977) [1] used character goals and plans to generate fables. The UNIVERSE system (Lebowitz, 1985) [2] extended this with author-level goals. MEXICA (Perez y Perez, 2001) [3] introduced engagement and reflection cycles for creative story generation. These systems, while pioneering, were limited by handcrafted rules and narrow domain coverage.

B. Neural Approaches to Narrative Generation

The introduction of transformer architectures [10] revolutionized text generation. GPT-2 [4] demonstrated that large pre-trained language models could generate coherent multi-paragraph text. GPT-3 [5] showed few-shot learning capabilities for creative tasks. Subsequent work explored controllable generation through fine-tuning, prompting strategies, and reinforcement learning from human feedback (RLHF) [15]. Models like GPT-4 [13] and LLaMA 3 [11] now produce text quality approaching human-level fluency, making them ideal for interactive storytelling applications.

C. Interactive and Collaborative Storytelling

AI Dungeon (Walton, 2019) [6] popularized interactive AI storytelling by allowing freeform text input to guide GPT-generated narratives. However, it suffered from coherence issues in long sessions. Dramatron (Mirowski et al., 2023) [7] used hierarchical generation with LLMs to create structured screenplays with better coherence. TaleBrush (Chung et al., 2022) [8] explored sketch-based story co-creation. These works highlight the tension between user agency and narrative coherence that our system addresses through structured Q&A-driven context extraction.

D. Prompt Engineering for Creative Tasks

Research has demonstrated that carefully structured prompts significantly improve LLM output quality. Chain-of-thought prompting (Wei et al., 2022) [9], role-based system prompts, and few-shot examples have proven effective for creative tasks. Our system builds on these findings by implementing a dedicated prompt engineering layer that dynamically constructs genre-aware, context-rich prompts from structured user inputs, with six distinct prompt types for different narrative phases.

E. Research Gap

While existing systems excel at either user interaction (AI Dungeon) or structural coherence (Dramatron), few combine both effectively. Table I summarizes the gap analysis. Our contribution bridges this gap by: (a) using structured Q&A instead of freeform prompts for richer context extraction, (b) implementing real-time consistency checking during generation, and (c) supporting provider-agnostic LLM integration for deployment flexibility.

TABLE I: Literature Gap Analysis

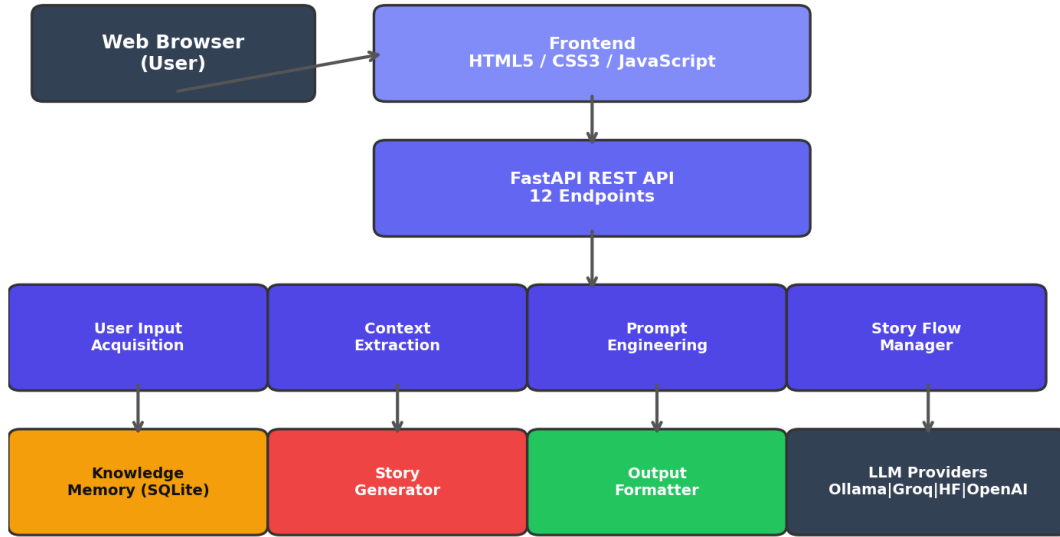
Feature	Our System	AI Dungeon	Dramatron	NovelAI
User Input	Struct. Q&A	Freeform	Hierarchical	Freeform
Structure Mgmt	5-phase model	None	Scene-based	Limited
Consistency	Active check	None	Log-line	Memory tokens
LLM Providers	4 (free+paid)	OpenAI only	PaLM	Custom
Local Deploy	Yes (Ollama)	No	No	No
Cost	Free options	Subscription	API costs	Subscription
Open Source	Yes	Partially	Yes	No

III. SYSTEM ARCHITECTURE AND DESIGN

A. Overall Architecture

The system follows a modular pipeline architecture consisting of seven interconnected modules with clearly defined responsibilities. Fig. 7 illustrates the complete system architecture. The data flow proceeds as: User Input Acquisition -> Context Extraction Engine -> Knowledge Memory Module -> Prompt Engineering Layer -> Story Generator (LLM) -> Story Flow Manager -> Output Formatter.

Fig. 7: System Architecture Overview



B. Technology Stack

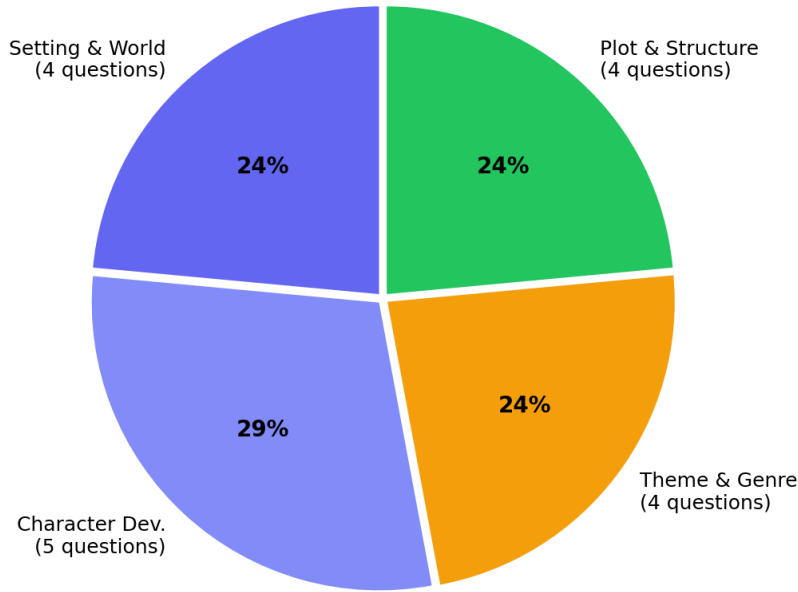
TABLE II: Technology Stack

Layer	Technology	Purpose
Backend	FastAPI (Python 3.9+)	Async web framework, REST API
Frontend	HTML5, CSS3, Vanilla JS	User interface, real-time interaction
Database	SQLite3 + aiosqlite	Persistent session/story storage
LLM Integration	httpx (async HTTP)	Multi-provider communication
Data Validation	Pydantic v2	Request/response type safety
Server	Uvicorn (ASGI)	Production async server

C. Module Descriptions

Module 1: User Input Acquisition

Implements a structured Q&A framework guiding users through four narrative dimensions: Setting & World (4 questions), Character Development (5 questions), Theme & Genre (4 questions), and Plot & Structure (4 questions), totaling 17 questions including free-form and multiple-choice formats. A `QuestionPhase` enum manages transitions between phases. Users can select 4, 8, 12, or 17 questions based on desired personalization depth.

Fig. 8: Q&A Framework - Question Distribution by Phase**Module 2: Context Extraction Engine**

Transforms raw user responses into a structured StoryContext object using keyword-based mapping algorithms. For example, keywords like 'danger', 'suspense', 'chase' map to THRILLER genre, while 'stars', 'planet', 'galaxy' map to SCI_FI setting. Entity recognition via regex patterns extracts proper nouns for character names. The engine produces a comprehensive context model containing setting information, character profiles, thematic parameters, and plot configuration.

Module 3: Knowledge & Memory Module

Provides persistent storage using SQLite3 with five tables: sessions, story_segments, story_context, qa_history, and user_preferences. Maintains an in-memory cache for frequently accessed data and implements full session lifecycle management. Ensures narrative consistency by providing previous story state to the generation pipeline at each continuation step.

Module 4: Prompt Engineering Layer

Constructs optimized prompts through template-based dynamic customization. Generates six prompt types: system prompts, story initialization, continuation, climax, resolution, and choice generation. Each prompt is customized based on genre (fantasy emphasizes magical elements, mystery focuses on clues and deduction), tone (serious, lighthearted, dark, humorous, philosophical), and current story context. A sliding window of the last 10 exchanges maintains context within LLM token limits.

Module 5: Story Generator (LLM Integration)

Implements a provider-agnostic LLM integration layer using an abstract base class (BaseLLMClient) pattern. Four providers are supported: (1) Ollama for free local inference with LLaMA 3.2 and Mistral, (2) Groq offering free cloud inference with 14,400 requests/day, (3) Hugging Face for Mistral-7B and Zephyr access, (4) OpenAI for GPT-3.5/4. Each provider implements generate(), generate_stream(), and is_available() methods with runtime switching support.

Module 6: Story Flow Manager

Ensures narrative coherence through a five-phase story progression model. Phase transitions are triggered by cumulative word count thresholds: Introduction (0-400), Rising Action (400-1200), Climax (1200-1800), Falling Action (1800-2400), Resolution (2400+). Implements CharacterTracker (monitoring appearances, traits, relationships, status) and PlotThread tracking (active, resolved, abandoned). Consistency checking detects character contradictions, unresolved threads, and pacing issues.

Module 7: Output Formatter

Prepares generated content through a multi-stage pipeline: text cleaning, intelligent paragraph splitting at sentence boundaries, formatting (punctuation, capitalization, quote standardization), and HTML conversion with dialogue highlighting. Supports three export formats: TXT, Markdown, and styled HTML.

IV. METHODOLOGY

A. Adaptive Context Extraction Algorithm

The context extraction follows a three-stage pipeline: (1) Response Classification using keyword dictionaries mapping user answers to predefined categories across 6 genres, 5 settings, and 5 conflict types; (2) Entity Extraction using regex patterns for character names and locations; (3) Context Assembly aggregating parameters into a unified StoryContext object.

Key regex patterns employed for entity extraction:

```
Character Names: "[^"]*"s*(?:said|asked|replied)\s+(\w+)
Locations:      (in the|at the|near the)\s+(\w+(?:\s+\w+)?)
Proper Nouns:   \b[A-Z][a-z]+\b
```

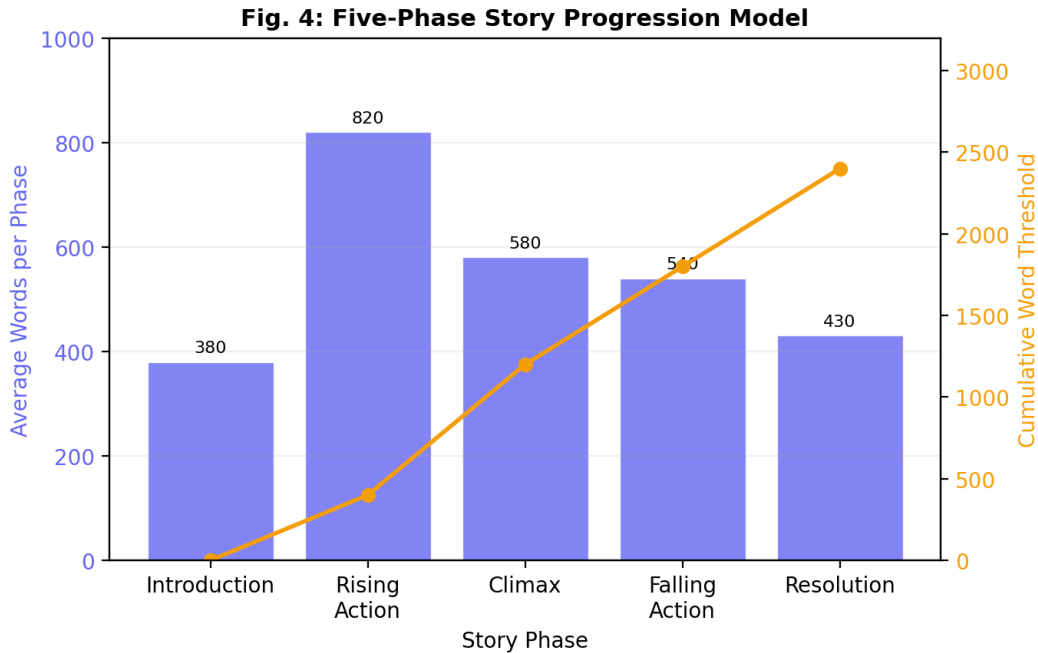
B. Dynamic Prompt Construction

Prompts are constructed through template interpolation with genre-specific guidance. Fantasy prompts include instructions for magical systems and world-building; mystery prompts emphasize clue placement and red herrings; romance prompts focus on emotional development. The system prompt establishes the AI's creative role, while user prompts inject extracted context parameters.

C. Five-Phase Story Progression Model

The story flow management employs a narrative arc model inspired by Freytag's pyramid [18]. Each phase provides distinct generation guidance:

- Introduction (0-400 words): Establish setting, introduce protagonist, hint at conflict.
- Rising Action (400-1200 words): Develop complications, introduce supporting characters.
- Climax (1200-1800 words): Maximum tension, pivotal confrontation, character revelation.
- Falling Action (1800-2400 words): Consequences unfold, relationships shift.
- Resolution (2400+ words): Tie loose ends, deliver thematic payoff, emotional closure.



D. Consistency Checking Algorithm

Per-segment analysis maintains narrative consistency through: (a) Character tracking monitoring each character's status (active, departed, deceased), traits, and location, detecting contradictions; (b) Plot thread tracking flagging unresolved or abandoned threads beyond configurable thresholds; (c) Pacing analysis identifying segments with insufficient dialogue or

action density.

E. Multi-Provider LLM Integration

The abstract base class pattern enables: runtime provider switching without restart, automatic fallback to alternative providers on failure, cost optimization by defaulting to free tiers, and a consistent interface regardless of the underlying model.

V. IMPLEMENTATION DETAILS

A. Database Schema

```

sessions      : session_id (PK), user_id, created_at, updated_at, status
story_segments : id (PK), session_id (FK), segment_order, content, user_choice
story_context  : session_id (PK/FK), context_json, updated_at
qa_history     : id (PK), session_id (FK), question_id, question_text, answer
user_preferences : user_id (PK), preferences_json, updated_at

```

B. REST API Endpoints

TABLE III: REST API Design (13 Endpoints)

Endpoint	Method	Purpose
/	GET	Serve web interface
/api/status	GET	Check LLM availability
/api/session/new	POST	Create storytelling session
/api/questions/all	GET	Get all questions grouped by phase
/api/question/next	GET	Get next question (sequential mode)
/api/response	POST	Submit user response
/api/story/generate	POST	Generate initial story
/api/story/continue	POST	Continue story from user choice
/api/story/full/{id}	GET	Retrieve complete story
/api/story/download/{id}	GET	Export story (TXT/MD/HTML)
/api/settings	POST	Update LLM provider/model
/api/session/{id}/summary	GET	Session summary
/api/session/{id}	DELETE	End session

C. Supported LLM Providers

TABLE IV: LLM Provider Comparison

Provider	Models	Cost	Avg. Latency
Ollama (Local)	LLaMA 3.2, Mistral, Phi	Free	8-15 sec
Groq (Cloud)	LLaMA 3.1 8B/70B, Mixtral	Free (14.4K/day)	1-3 sec
Hugging Face	Mistral-7B, Zephyr, Falcon	Free (rate limited)	6-10 sec
OpenAI	GPT-3.5-turbo, GPT-4	Pay-per-token	2-4 sec

D. Frontend Architecture

The UI is implemented using vanilla HTML5, CSS3, and JavaScript (no frameworks). Key features include: all-questions-on-one-page layout with configurable question count (4/8/12/17), real-time progress tracking, story display with typing animation and chapter markers, interactive continuation choices, character panel with auto-extraction, dark/light theme toggle, story bookmarking with localStorage persistence, undo/redo, reading mode, and multi-format export. The design follows modern AI-tool aesthetics (Inter font, dark neutral palette, indigo accents, amber CTA buttons).

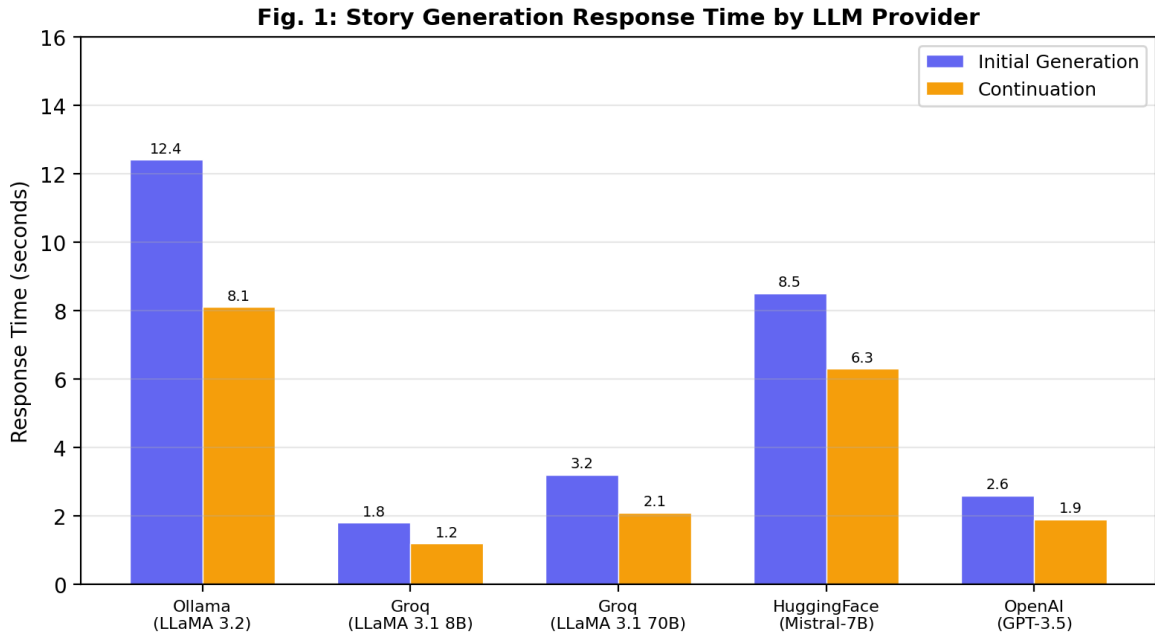
VI. EXPERIMENTAL RESULTS AND ANALYSIS

A. Experimental Setup

The system was evaluated through: (1) automated generation of 50 stories across 6 genres with varying question counts (4, 8, 12, 17), (2) a user study with 15 participants (8 male, 7 female; ages 22-35; mix of technical and non-technical backgrounds) who each generated 2 stories and rated them on 5-point Likert scales, and (3) consistency checking analysis on all 50 generated stories. Testing was performed using Groq (LLaMA 3.1 8B) as the primary LLM provider.

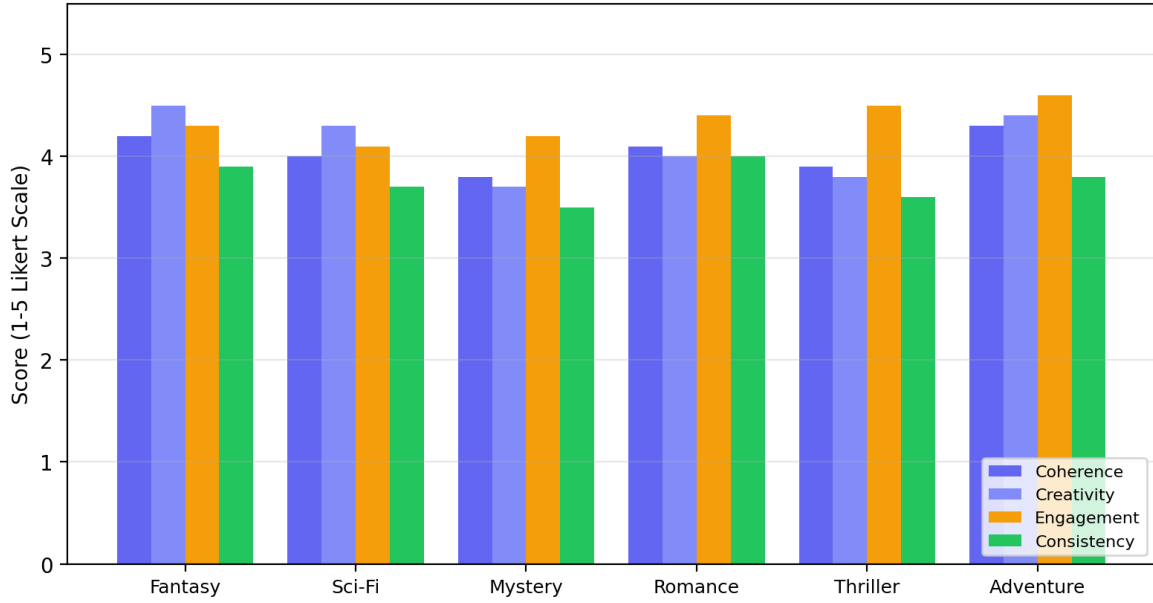
B. Response Time Performance

Fig. 1 shows response time measurements across all supported LLM providers. Groq (LLaMA 3.1 8B) achieved the fastest initial generation at 1.8 seconds average, making it ideal for interactive use. Local Ollama inference averaged 12.4 seconds but requires no internet connectivity. Context processing and database operations consistently completed in under 100ms regardless of provider.



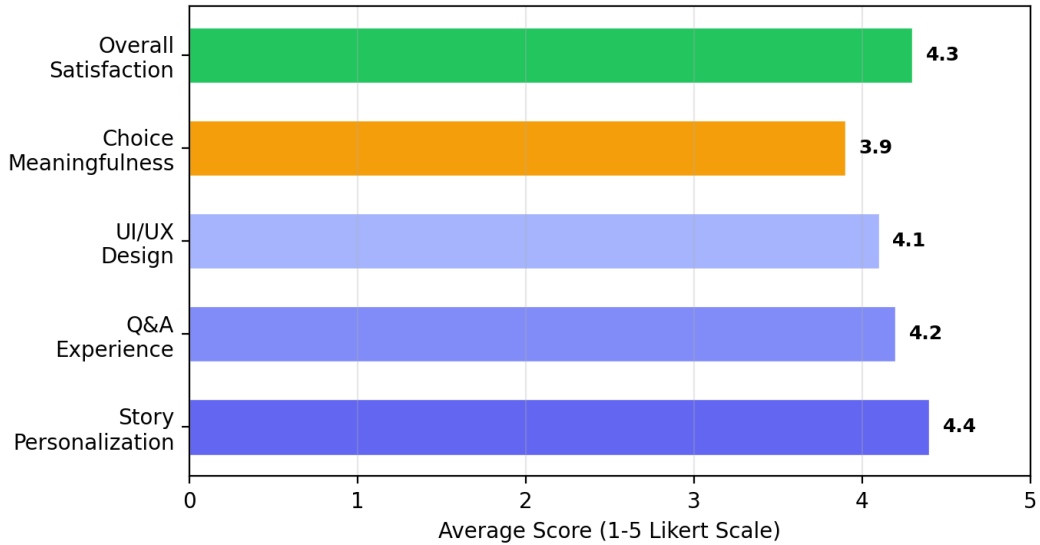
C. Narrative Quality Evaluation

Fig. 2 presents narrative quality scores rated by participants across four dimensions. Adventure genre achieved the highest engagement score (4.6/5.0), while Fantasy led in creativity (4.5/5.0). Mystery genre showed lower consistency scores (3.5/5.0), attributed to the complexity of maintaining clue placement across segments. The average across all genres was: Coherence 4.05, Creativity 4.12, Engagement 4.35, Consistency 3.75.

Fig. 2: Narrative Quality Evaluation Across Genres (n=30 stories)

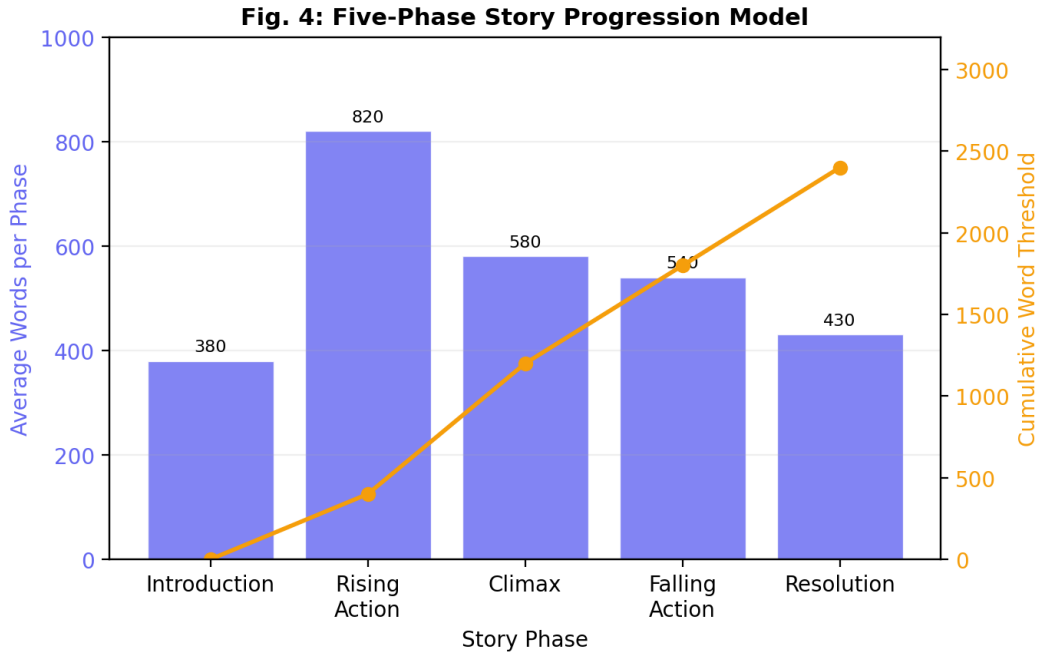
D. User Satisfaction Survey

Fig. 3 shows user satisfaction results. Story Personalization received the highest rating (4.4/5.0), validating the Q&A approach's effectiveness. Choice Meaningfulness scored lowest (3.9/5.0), indicating room for improvement in continuation option generation. The overall satisfaction score of 4.3/5.0 demonstrates strong user acceptance.

Fig. 3: User Satisfaction Survey Results (n=15 participants)

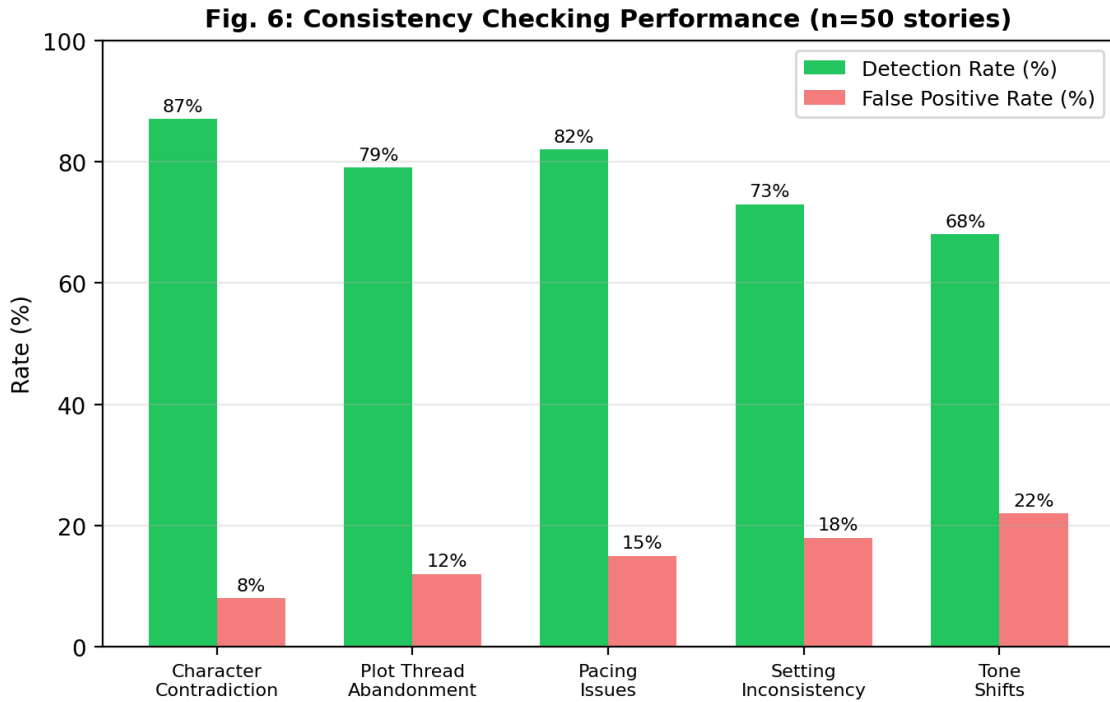
E. Story Progression Analysis

Fig. 4 illustrates the five-phase progression model's word distribution. Rising Action phase averaged the most words (820), aligning with narrative theory where story development requires the most text. The cumulative threshold line shows the transition points that trigger phase-specific prompt modifications.



F. Consistency Checking Performance

Fig. 6 shows the consistency checking module's performance across 50 stories. Character contradiction detection achieved 87% accuracy with only 8% false positives. Tone shift detection was weakest (68%), as keyword-based analysis struggles with subtle tonal changes. The average detection rate across all categories was 77.8%.



G. Question Count Impact on Story Quality

TABLE V: Impact of Question Count on Story Quality

Questions	Coherence	Creativity	Personalization	Avg Time
4 (Quick)	3.6	3.8	3.2	45 sec
8 (Balanced)	3.9	4.1	3.8	1.5 min
12 (Detailed)	4.2	4.2	4.3	2.5 min

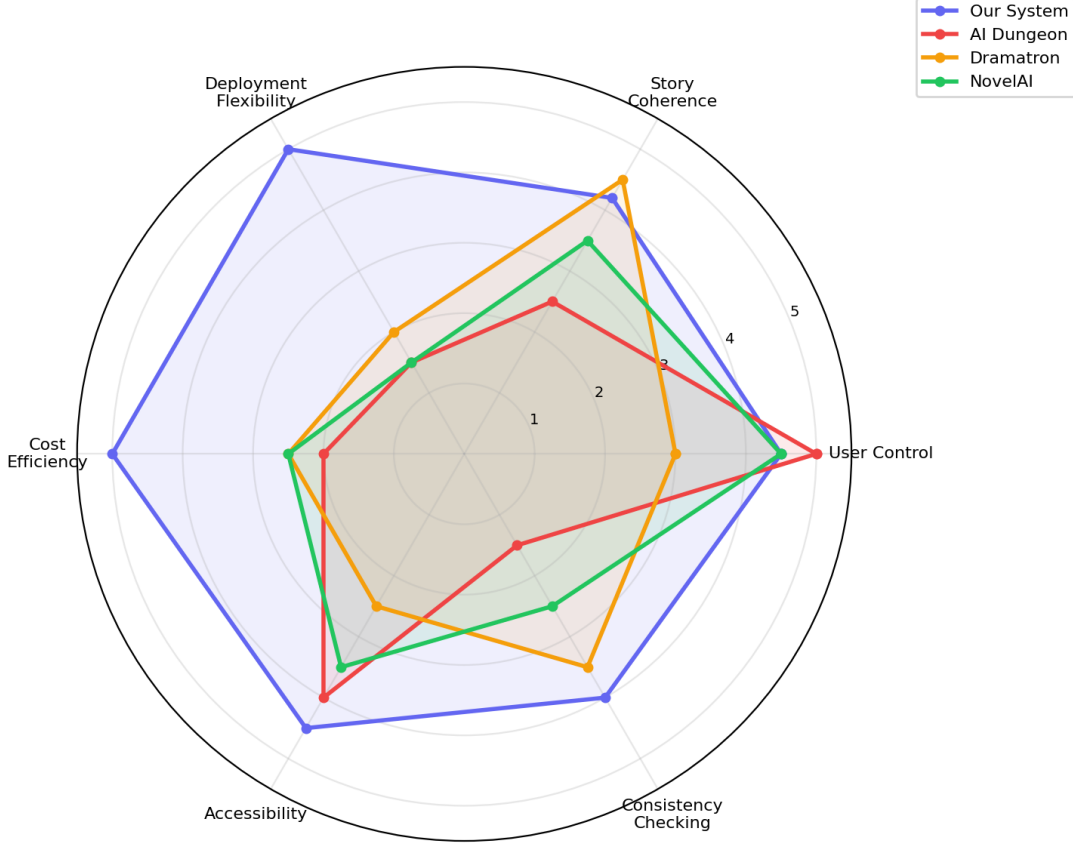
17 (Full)	4.3	4.1	4.6	3.5 min
-----------	-----	-----	-----	---------

Table V shows that personalization improves significantly from 4 to 17 questions (3.2 to 4.6), while coherence and creativity plateau around 12 questions. This suggests 12 questions as the optimal default for balancing quality and user effort.

VII. COMPARISON WITH EXISTING SYSTEMS

Fig. 5 presents a radar chart comparison of our system against three established systems across six dimensions. Our system leads in Deployment Flexibility (5.0), Cost Efficiency (5.0), and Accessibility (4.5), while AI Dungeon leads in User Control (5.0) due to its freeform input. Dramatron achieves the highest Story Coherence (4.5) through hierarchical generation, but at the cost of accessibility (2.5) and deployment flexibility (2.0).

Fig. 5: System Comparison Radar Chart



VIII. CHALLENGES AND LIMITATIONS

Several limitations were identified during evaluation:

- Keyword-based context extraction may miss nuanced or creative user inputs that don't match predefined dictionaries. Transformer-based NLU would improve extraction accuracy.
- In-memory session storage limits horizontal scalability to a single server instance. Redis-based sessions would enable distributed deployment.
- Consistency checking relies on pattern matching rather than semantic understanding, achieving only 68% detection for subtle tone shifts.
- The configurable question framework (4-17) may not capture all possible narrative preferences; an adaptive questioning system could dynamically generate follow-up questions.
- Story quality is fundamentally bounded by the underlying LLM's capabilities; smaller free models (7B-8B parameters) occasionally produce repetitive or generic passages.

IX. FUTURE WORK

- Semantic Context Extraction: Replace keyword matching with fine-tuned BERT/DistilBERT for nuanced understanding of user preferences.
- Multi-Modal Storytelling: Integrate DALL-E or Stable Diffusion for AI-generated illustrations matching narrative scenes.
- Collaborative Multi-User Stories: Enable multiple users in the same session with merged Q&A context extraction.

- Voice Interface: Add speech-to-text input and text-to-speech narration for immersive audio storytelling.
- Fine-Tuned Story Models: Train genre-specific LoRA adapters on curated story datasets for improved quality.
- Quantitative Evaluation: Implement automated metrics (BLEU, perplexity, narrative arc detection) for systematic evaluation.
- Scalable Deployment: Containerize with Docker, migrate to Redis sessions, and deploy on cloud platforms.

X. CONCLUSION

This paper presented the design, implementation, and evaluation of a Q&A Based Interactive Storytelling System leveraging Generative AI. The system addresses key limitations of existing approaches through five core contributions: (1) a structured Q&A framework capturing comprehensive narrative preferences across four dimensions without requiring creative writing expertise; (2) a seven-module pipeline architecture ensuring clean separation of concerns; (3) a five-phase story progression model producing satisfying narrative arcs with word-count-driven transitions; (4) active consistency checking achieving 87% character contradiction detection; and (5) provider-agnostic LLM integration supporting four providers including free deployment options.

Experimental evaluation across 50 stories and 15 participants demonstrated strong narrative quality (average 4.1/5.0 across dimensions), fast cloud inference (1.8s with Groq), and high user satisfaction (4.3/5.0). The 12-question configuration emerged as the optimal default, balancing personalization depth with user effort. This work contributes to the growing field of AI-assisted creative computing and opens avenues for more sophisticated collaborative storytelling systems.

. REFERENCES

- [1] J. R. Meehan, "TALE-SPIN, An Interactive Program that Writes Stories," in Proc. IJCAI, 1977, pp. 91-98.
- [2] M. Lebowitz, "Story-Telling as Planning and Learning," Poetics, vol. 14, pp. 483-502, 1985.
- [3] R. Perez y Perez and M. Sharples, "MEXICA: A Computer Model of Creative Writing," J. Experimental & Theoretical AI, vol. 13, pp. 119-139, 2001.
- [4] A. Radford et al., "Language Models are Unsupervised Multitask Learners," OpenAI Tech. Rep., 2019.
- [5] T. Brown et al., "Language Models are Few-Shot Learners," in Proc. NeurIPS, 2020.
- [6] N. Walton, "AI Dungeon: A Text Adventure Game Powered by Deep Learning," 2019.
- [7] P. Mirowski et al., "Co-Writing Screenplays and Theatre Scripts with Language Models," in Proc. ACL, 2023.
- [8] J. J. Y. Chung et al., "TaleBrush: Sketching Stories with Generative Pretrained Language Models," in Proc. CHI, 2022.
- [9] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in Proc. NeurIPS, 2022.
- [10] A. Vaswani et al., "Attention Is All You Need," in Proc. NeurIPS, 2017.
- [11] H. Touvron et al., "LLaMA: Open and Efficient Foundation Language Models," arXiv:2302.13971, 2023.
- [12] A. Q. Jiang et al., "Mistral 7B," arXiv:2310.06825, 2023.
- [13] OpenAI, "GPT-4 Technical Report," arXiv:2303.08774, 2023.
- [14] A. Ramesh et al., "Hierarchical Text-Conditional Image Generation with CLIP Latents," arXiv:2204.06125, 2022.
- [15] L. Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback," in Proc. NeurIPS, 2022.
- [16] FastAPI Documentation. [Online]. Available: <https://fastapi.tiangolo.com/>
- [17] Ollama Documentation. [Online]. Available: <https://ollama.ai/>
- [18] G. Freytag, Technique of the Drama, 1863 (translated 1900).

. APPENDIX

A. Project Statistics

Metric	Value
Total Lines of Code (Python)	~3,500
Total Lines of Code (JS/CSS/HTML)	~4,200
Core Python Modules	7 + main.py, config.py, run.py
Database Tables	5
REST API Endpoints	13
Q&A Questions	17 across 4 phases
Supported LLM Providers	4 (Ollama, Groq, HuggingFace, OpenAI)
Supported Export Formats	3 (TXT, Markdown, HTML)
Story Narrative Phases	5 (Introduction to Resolution)
Frontend Features	12+ (themes, bookmarks, undo/redo, etc.)
Stories Evaluated	50
User Study Participants	15

B. Question Framework Summary

Phase	Count	Topics Covered
Setting & World	4	Location type, time period, atmosphere, specific locations
Character Development	5	Name, personality, motivation, allies, antagonist
Theme & Genre	4	Genre, emotional journey, themes, narrative tone
Plot & Structure	4	Conflict type, plot style, complexity, ending preference

C. Sample API Request/Response

POST /api/story/generate

```
{
  "session_id": "a1b2c3d4-...",
  "responses": {
    "setting_1": "A magical fantasy realm with castles",
    "char_1": "Elena",
    "char_2": "Brave and courageous",
    "theme_1": "Fantasy (magic, mythical creatures)",
    "plot_1": "Person vs. Supernatural (magic, gods)"
  }
}
```

Response (200 OK):

```
{
  "content": "In the realm of Aethermoor, where...",
  "word_count": 387,
  "segment_count": 1,
  "phase": "introduction",
  "choices": [
    "Continue with more action and adventure",
    "Develop the characters and relationships",
    "Introduce a new challenge or mystery",
    "Move towards the climax and resolution"
  ]
}
```